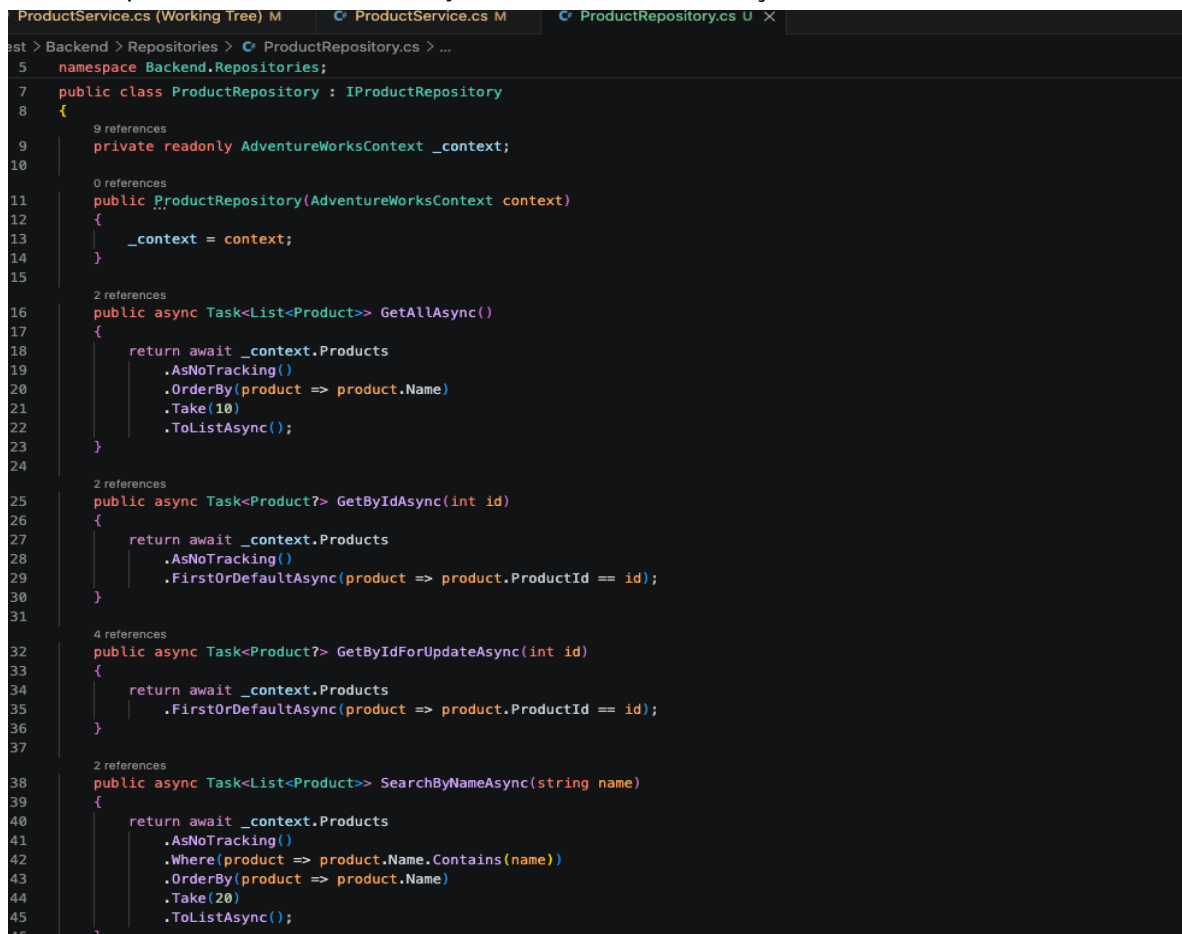


Reporte de Actividades 13

1. Repository Pattern
2. Manejo global de errores
3. Middleware
4. ProblemDetails
5. ILogger

Repository

Un repositorio es una clase que se encarga de lo relacionado con la base de datos, es el que obtiene por así decirlo los datos en crudo, y ahí es donde el servicio los procesa, y el controlador le da formato con las respuestas http todos estos conceptos ya los había utilizado antes, pero volver retomarlos me esta gustando bastante, por lo acelerado que había estado haciendo proyectos anteriormente mas que nada por necesidad había dejado de lado todo esto



```
ProductService.cs (Working Tree) M | ProductService.cs M | ProductRepository.cs U X
Backend > Repositories > ProductRepository.cs > ...
5 namespace Backend.Repositories;
7 public class ProductRepository : IProductRepository
8 {
9     9 references
10     private readonly AdventureWorksContext _context;
11
12     0 references
13     public ProductRepository(AdventureWorksContext context)
14     {
15         _context = context;
16     }
17
18     2 references
19     public async Task<List<Product>> GetAllAsync()
20     {
21         return await _context.Products
22             .AsNoTracking()
23             .OrderBy(product => product.Name)
24             .Take(10)
25             .ToListAsync();
26     }
27
28     2 references
29     public async Task<Product?> GetByIdAsync(int id)
30     {
31         return await _context.Products
32             .AsNoTracking()
33             .FirstOrDefaultAsync(product => product.ProductId == id);
34     }
35
36     4 references
37     public async Task<Product?> GetByIdForUpdateAsync(int id)
38     {
39         return await _context.Products
40             .FirstOrDefaultAsync(product => product.ProductId == id);
41     }
42
43     2 references
44     public async Task<List<Product>> SearchByNameAsync(string name)
45     {
46         return await _context.Products
47             .AsNoTracking()
48             .Where(product => product.Name.Contains(name))
49             .OrderBy(product => product.Name)
50             .Take(20)
51             .ToListAsync();
52     }
53 }
```

Ahora todo lo que tenia que ver con la base de datos e abstraigo con este repositorio y en el servicio todo se ve aun mas limpio y bonito

Tambien agregue la interfaz para el repositorio para seguir el mismo patron

```
ProductService.cs (Working Tree) M | ProductService.cs M X | ProductRepository.cs U
est > Backend > Services > ProductService.cs > ...
6 namespace Backend.Services;
8 public class ProductService : IProductService
17 //task es para que sea asincrono el metodo
18     2 references
19     public async Task<List<ProductDto>> GetAllAsync()
20     {
21         var products = await _productRepository.GetAllAsync();
22         return products.Adapt<List<ProductDto>>();
23     }
24     2 references
25     public async Task<ProductDetailDto?> GetByIdAsync(int id)
26     {
27         var product = await _productRepository.GetByIdAsync(id);
28         return product.Adapt<ProductDetailDto?>();
29     }
30     //task es para que sea asincrono
31     2 references
32     public async Task<List<ProductDto>> SearchByNameAsync(string name)
33     {
34         var products = await _productRepository.SearchByNameAsync(name);
35         return products.Adapt<List<ProductDto>>();
36     }
37     2 references
38     public async Task<(ProductWriteResult Result, ProductDetailDto? Product)> CreateAsync(CreateProductDto productDto)
39     {
40         var productNumberExists = await _productRepository.ProductNumberExistsAsync(productDto.ProductNumber);
41         if (productNumberExists)
42         {
43             return (ProductWriteResult.Conflict, null);
44         }
45         var product = productDto.Adapt<Product>();
46         product.SellStartDate = DateTime.UtcNow;
47         product.ModifiedDate = DateTime.UtcNow;
48         await _productRepository.AddAsync(product);
49         await _productRepository.SaveChangesAsync();
50         return (ProductWriteResult.Success, product.Adapt<ProductDetailDto>());
51     }
52     2 references
53     public async Task<ProductWriteResult> UpdateAsync(int id, UpdateProductDto productDto)
54     {
55     }
56
ProductService.cs (Working Tree) M | ProductService.cs M | IProductRepository.cs U X | ProductRepository.cs U
est > Backend > Repositories > IProductRepository.cs > ...
1 using Backend.Models;
2
3 namespace Backend.Repositories;
4
5 4 references
6 public interface IProductRepository
7 {
8     2 references
9     Task<List<Product>> GetAllAsync();
10    2 references
11    Task<Product?> GetByIdAsync(int id);
12    4 references
13    Task<Product?> GetByIdForUpdateAsync(int id);
14    2 references
15    Task<List<Product>> SearchByNameAsync(string name);
16    4 references
17    Task<bool> ProductNumberExistsAsync(string productNumber, int? excludedProductId = null);
18    2 references
19    Task AddAsync(Product product);
20    2 references
21    void Delete(Product product);
22    5 references
23    Task SaveChangesAsync();
24 }
```

Manejo Global de Errores y Middleware

Estos sirven en conjunto para poder cachar cualquier error inesperado que pueda ocurrir al momento de hacer una petición http, ese se usa de manera general, no importa que método

Aquí la clave este en el `_next`, que dice que siga con lo que estaba haciendo, pero si hay un error lo cachea y hace cosas

ProblemDetails

Es un formato estándar que tiene .net para escribir errores http haciendo que se describa de mejor manera

ILogger

El `ILogger` sirve para nosotros como desarrolladores encontrar los problemas inesperados de manera mas rápida

Aquí me salieron los tipos de logs pero por el momento solo use uno que es el de `logError`

Nivel	Qué significa	Ejemplo
Trace	Detalle extremo	Paso por paso de algo interno
Debug	Información para desarrollar	Valor de una variable mientras pruebas
Information	Algo normal ocurrió	Se creó un producto
Warning	Algo raro, pero la app sigue	Producto no encontrado muchas veces
Error	Algo falló	Excepción al guardar en base de datos
Critical	Fallo grave	La app no puede conectarse a la BD

test > Backend > Middleware > GlobalExceptionMiddleware.cs > ...

```
4 namespace Backend.Middleware;
6 public class GlobalExceptionMiddleware

    2 references
8     private readonly RequestDelegate _next;
    2 references
9     private readonly ILogger<GlobalExceptionMiddleware> _logger;
10
    0 references
11     public GlobalExceptionMiddleware(
12         RequestDelegate next,
13         ILogger<GlobalExceptionMiddleware> logger)
14     {
15         _next = next;
16         _logger = logger;
17     }
18
    0 references
19     public async Task InvokeAsync(HttpContext context)
20     {
21         try
22         {
23             await _next(context);
24         }
25         catch (Exception exception)
26         {
27             var statusCode = (int)HttpStatusCode.InternalServerError;
28
29             _logger.LogError(
30                 exception,
31                 "Ocurrio un error inesperado al procesar {Method} {Path}",
32                 context.Request.Method,
33                 context.Request.Path);
34
35             context.Response.StatusCode = statusCode;
36             context.Response.ContentType = "application/problem+json";
37
38             var problemDetails = new ProblemDetails
39             {
40                 Status = statusCode,
41                 Title = "Error interno del servidor",
42                 Detail = "Ocurrio un error inesperado.",
43                 Instance = context.Request.Path
44             };
45
46             await context.Response.WriteAsJsonAsync(problemDetails);
47         }
48     }
```